

Multiple cross-site leaks disclosing Facebook users in third-party websites

Multiple cross-site leaks disclosing Facebook users in third-party websites -

Youssef Sammouda, Youssef Sammouda (sam0) personal blog.

- Published: 2026-01-16
- Original: <<https://ysamm.com/uncategorized/2026/01/16/cross-site-leaks.html>>
- Preserved from: <https://ysamm.com/uncategorized/2026/01/16/cross-site-leaks.html> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Introduction

This write-up consolidates several XS-Leak issues discovered across Meta-owned platforms, including Facebook, Workplace, Meta for Work, and internal Meta surfaces.

While none of these bugs directly grant account control on their own, they enable precise user identification, de-anonymization, and cross-context fingerprinting. When chained with other vulnerabilities, these primitives significantly lower the bar for targeted attacks, account takeover, and employee profiling.

Each section below documents a distinct class of XS-Leak, grouped by affected surface and leakage primitive.

XS-Leak: Identifying the Current Workplace / Meta Work User

This issue allows distinguishing the currently logged-in Workplace user by abusing response-type differences in Zoom callback endpoints.

By loading the following endpoints in a script context:

- <https://work-api.workplace.com/zoom/callback/>
- <https://work-api.facebook.com/zoom/callback/>

and supplying both `__cid` (community ID) and `__user` parameters, an attacker can brute-force the user ID within a known Workplace community.

When `__user` is correct for the active session, the endpoint responds with an empty page and `Content-Type: text/html`, which can be loaded successfully via a script tag. When incorrect, the response is `application/json`, triggering CORB and blocking script execution.

By observing `onload` versus `onerror` events, the attacker can reliably determine whether a given user ID matches the currently logged-in Workplace user.

This works across all major browsers for Workplace. For Facebook users, the same technique only works inside the Facebook app webview, where `__cid` is not properly validated.

XS-Leak: Identifying the Active Facebook User or Page

This issue abuses Facebook plugin behavior to distinguish the currently logged-in Facebook user or page.

By embedding a Facebook plugin (such as the Like plugin) inside an iframe and supplying both `__a=1` and a candidate `__user` ID, different behavior is observed depending on whether the supplied ID matches the active session.

If the `__user` value is correct, the iframe successfully loads and sends a `postMessage` to the parent window. If incorrect, the response includes `X-Frame-Options: deny`, preventing the iframe from loading.

This distinction allows brute-forcing the active user or page ID by observing `postMessage` events versus timeouts.

This technique works for both personal profiles and pages and does not require any special permissions beyond the victim being logged into Facebook.

PoC

```

<html>
<body>
<iframe width="0" height="0" id="test" src=""></iframe>
<script>
const tryUserID = (USER_ID) =>{
    frm_id = document.getElementById("test");
    frm_id.src = `https://www.facebook.com/plugins/like.php?
app_id=&container_width=0&href=https%3A%2F%2Fysamm.com&layout=button_count&locale=fr_FR&s
dk=joey&send=true&show_faces=false&width=450&channel_url=https%3A%2F%2Fstaticxx.facebook.
com%2F%2Fconnect%2Fxd_arbiter%2F%3Fversion%3D46%23cb%3Dcc%26is_canvas%3Dtrue%26origin%3D
https%253A%252F%252Fysamm.com%26relation%3Dopener&__a=1&__user=${Number(USER_ID)}`;
    return new Promise((resolve) => {
        onmessage = (e) => {
            resolve(true);
        }
        setTimeout(function(){
            resolve(false)
        },2000)
    });
}
onhashchange = () => {
    user_id = window.location.hash.split("#")[1]
    tryUserID(user_id).then((e) => {alert(e)})
}
</script>
</body>
</html>

```

XS-Leak: Facebook User ID Leakage via signals/iwl.js

This issue enables direct leakage of the Facebook user ID on third-party origins.

The endpoint `https://www.facebook.com/signals/iwl.js?pixel_id=PIXEL_ID` returns a JavaScript payload intended for internal Meta Pixel testing, which includes the current Facebook user ID.

Although the value is scoped inside a function, it can be extracted by abusing JavaScript prototype manipulation prior to loading the script.

Unlike other XS-Leaks, this technique does not rely on brute-forcing or timing side channels and yields an exact user identifier.

This allows full de-anonymization of Facebook users on third-party websites, enabling cross-site tracking, targeted attacks, and precise user correlation.

PoC

```
<html>
<head>
<script>Function.prototype.default = (e)=>{
  if (typeof e.userID == "string") alert(e.userID)
};Function.prototype.__esModule = 1;</script>
</head>
<body>
<script src="https://www.facebook.com/signals/iwl.js?pixel_id=PIXEL_ID"></script>
</body>
</html>
```

The endpoint is only accessible when the user is logged in, making login state detection implicit as well.

XS-Leak: Employee and Login State Identification Across Meta Properties

This class of issues enables leaking multiple attributes about the current attacker website visitor, including:

- Whether the user is logged into Facebook
- Whether the user is logged into Workplace
- Whether the user is logged into work.meta.com
- Whether the user is a Meta employee
- The employee's Unix username or email prefix

The attack relies on probing Meta-owned subdomains such as:

- sb.facebook.com
- sb.internalfb.com
- my-od.facebook.com
- my-od.internalfb.com

Employee identifiers are reflected in subdomains (for example, www.username.sb.facebook.com). By brute-forcing subdomains and observing load behavior, an attacker can identify the currently logged-in employee.

For example:

```
https://www.testemployee.sb.facebook.com/random
https://www.testemployee.sb.internalfb.com/random
```

This can be combined with XS-Leak primitives (script loading, iframe behavior, or CORB differences) to confirm employee identity.

Additionally, because the `/signals/iwl.js` endpoint only loads when logged in, it can be used as a login-state oracle and combined with other probes to build a detailed fingerprint of the user across Meta platforms.

Impact

Collectively, these XS-Leaks enable:

- Identification of logged-in Facebook, Workplace, and Meta Work users
- Precise user ID de-anonymization on third-party websites
- Detection of Meta employees and internal account attributes
- Cross-platform fingerprinting without user interaction

Timeline

Dec 29, 2024 — Bug #1 reported

Dec 27, 2024 — Bug #2 reported

Dec 5, 2024 — Bug #3 reported

Dec 29, 2024 — Bug #4 reported

Dec 29, 2024 — Bugs Acknowledged by Meta

May 22, 2025 — Bugs fixed by Meta

Feb 12, 2025 — **\$2,400** bounty awarded by Meta for Bug #1

Feb 12, 2025 — **\$2,400** bounty awarded by Meta for Bug #2

Feb 12, 2025 — **\$3,600** bounty awarded by Meta for Bug #3

Feb 12, 2025 — Meta marked Bug #4 as informative

Archived from <https://ysamm.com/uncategorized/2026/01/16/cross-site-leaks.html>. Rendered offline from the local Markdown copy.